DATABASE TECHNOLOGIES LAB SHEET – 8

Course Code: MCA105P

Topics: Functions and Stored Procedures

Software: MySQL

PART A – DATABASE SETUP

Question: Create a database named Hospital_Functions_Lab and create Patients and Medicines tables with the given records.

SQL Code:

```
CREATE DATABASE Hospital_Functions_Lab;

USE Hospital_Functions_Lab;
s
CREATE TABLE Patients(
   PatientID INT PRIMARY KEY,
   PatientName VARCHAR(50),
   Age INT,
   Gender VARCHAR(10),
   WeightKg DECIMAL(5,2),
   ConsultationFee DECIMAL(10,2)
);

INSERT INTO Patients VALUES
(201,'Asha',24,'F',55.5,600),
(202,'Rohan',35,'M',72.0,800),
(203,'Meera',29,'F',61.5,700),
(204,'Vivek',46,'M',85.0,1000),
(205,'Nitin',17,'M',58.0,500),
(206,'Kavya',52,'F',68.0,900);

CREATE TABLE Medicines(
   MedicineID INT PRIMARY KEY,
   MedicineName VARCHAR(50),
   UnitPrice DECIMAL(10,2),
   Stock INT
);

INSERT INTO Medicines VALUES
(301,'Paracetamol',25,100),
(302,'Amoxicillin',80,50),
(303,'Cetirizine',35,75),
(304,'Omeprazole',60,40),
(305,'Azithromycin',120,25);

SELECT * FROM Patients;

SELECT * FROM Medicines;
```

PART B – FUNCTIONS

Q1. Calculate BMI

Question: Create a function calculate_bmi(weight_kg, height_m) that returns BMI. Test it with suitable values and record the result.

SQL Code:

```
DELIMITER //

CREATE FUNCTION calculate_bmi(
   weight_kg DECIMAL(5,2),
   height_m DECIMAL(5,2)
)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
   DECLARE bmi DECIMAL(5,2);

   SET bmi = weight_kg / (height_m * height_m);

   RETURN bmi;
END //

DELIMITER ;

SELECT calculate_bmi(60, 1.65) AS BMI;
```

Q2. Age Category

Question: Create a function age_category(age) that returns Child, Adult, or Senior according to rules you define. Apply it to every patient.

SQL Code:

```
DELIMITER //

CREATE FUNCTION age_category(p_age INT)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
   IF p_age < 18 THEN
      RETURN 'Child';

   ELSEIF p_age < 60 THEN
      RETURN 'Adult';

   ELSE
      RETURN 'Senior';
   END IF;
END //

DELIMITER ;

SELECT PatientID, PatientName, Age,
age_category(Age) AS AgeCategory
FROM Patients;
```

Q3. Consultation Fee with Tax

Question: Create a function consultation_with_tax(fee, tax_percent) that returns the consultation fee after adding the supplied tax percentage. Display original and calculated fees.

SQL Code:

```
DELIMITER //

CREATE FUNCTION consultation_with_tax(
   fee DECIMAL(10,2),
   tax_percent DECIMAL(5,2)
)
RETURNS DECIMAL(10,2)
```

```
DETERMINISTIC
BEGIN
   RETURN fee + (fee * tax_percent / 100);
END //

DELIMITER ;

SELECT PatientName,
ConsultationFee AS OriginalFee,
consultation_with_tax(ConsultationFee, 10)
AS FeeWithTax
FROM Patients;
```

Q4. Medicine Stock Value

Question: Create a function medicine_stock_value(unit_price, stock) that returns UnitPrice × Stock. Display the stock value for every medicine.

SQL Code:

```
DELIMITER //

CREATE FUNCTION medicine_stock_value(
   unit_price DECIMAL(10,2),
   p_stock INT
)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
   RETURN unit_price * p_stock;
END //

DELIMITER ;

SELECT MedicineID, MedicineName,
UnitPrice, Stock,
medicine_stock_value(UnitPrice, Stock)
AS StockValue
FROM Medicines;
```

PART C – STORED PROCEDURES

Q5. Patient by Gender

Question: Create a stored procedure patient_by_gender(IN p_gender) that displays patients of the supplied gender. Execute it for F and M.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE patient_by_gender(
   IN p_gender VARCHAR(10)
)
BEGIN
   SELECT *
   FROM Patients
   WHERE Gender = p_gender;
END //

DELIMITER ;

CALL patient_by_gender('F');
```

```
CALL patient_by_gender('M');
```

Q6. Patients Above Age

Question: Create a stored procedure patients_above_age(IN p_age) that displays patients older than the supplied age. Test it with at least two different ages.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE patients_above_age(
   IN p_age INT
)
BEGIN
   SELECT *
   FROM Patients
   WHERE Age > p_age;
END //

DELIMITER ;

CALL patients_above_age(25);

CALL patients_above_age(40);
```

Q7. Update Medicine Stock

Question: Create a stored procedure update_medicine_stock(IN p_id, IN p_quantity) that increases stock for the selected medicine. Execute it and verify the changed row.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE update_medicine_stock(
   IN p_id INT,
   IN p_quantity INT
)
BEGIN
   UPDATE Medicines
   SET Stock = Stock + p_quantity
   WHERE MedicineID = p_id;
END //

DELIMITER ;

CALL update_medicine_stock(301, 20);

SELECT * FROM Medicines
WHERE MedicineID = 301;
```

Q8. Count Total Medicines

Question: Create a stored procedure medicine_count(OUT p_count) that returns the total number of medicines. Call it using a session variable and display the result.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE medicine_count(
   OUT p_count INT
```

```
)
BEGIN
   SELECT COUNT(*)
   INTO p_count
   FROM Medicines;
END //

DELIMITER ;

CALL medicine_count(@total);

SELECT @total AS TotalMedicines;
```

Q9. Patient Report Using Functions

Question: Create a stored procedure patient_report() that displays PatientID, PatientName, Age, BMI using your BMI function, and age category using your age_category function.

SQL Code:

```
-- Add height column because it is not
-- available in the supplied Patients table.

ALTER TABLE Patients
ADD HeightM DECIMAL(5,2);

-- Sample heights for testing.
-- Replace these with actual patient heights.

UPDATE Patients SET HeightM = 1.60
WHERE PatientID = 201;

UPDATE Patients SET HeightM = 1.75
WHERE PatientID = 202;

UPDATE Patients SET HeightM = 1.65
WHERE PatientID = 203;

UPDATE Patients SET HeightM = 1.80
WHERE PatientID = 204;

UPDATE Patients SET HeightM = 1.68
WHERE PatientID = 205;

UPDATE Patients SET HeightM = 1.62
WHERE PatientID = 206;

DELIMITER //

CREATE PROCEDURE patient_report()
BEGIN
   SELECT
      PatientID,
      PatientName,
      Age,
      calculate_bmi(WeightKg, HeightM) AS BMI,
      age_category(Age) AS AgeCategory
   FROM Patients;
END //

DELIMITER ;

CALL patient_report();
```

Q10. Medicine Report Using Functions

Question: Create a stored procedure medicine_report() that displays MedicineID, MedicineName, UnitPrice, Stock, stock value using your medicine_stock_value function, and a readable stock status. Execute and record the complete report.

SQL Code:

```
DELIMITER //

CREATE PROCEDURE medicine_report()
BEGIN
   SELECT
      MedicineID,
      MedicineName,
      UnitPrice,
      Stock,

      medicine_stock_value(UnitPrice, Stock)
      AS StockValue,

      CASE
         WHEN Stock = 0 THEN 'Out of Stock'
         WHEN Stock <= 30 THEN 'Low Stock'
         ELSE 'In Stock'
      END AS StockStatus

   FROM Medicines;
END //

DELIMITER ;

CALL medicine_report();
```